Data Cleaning Using Apache Spark: Missing Data

Explore how to handle missing data in datasets using Apache Spark by understanding different types of missing data such as completely at random, at random, and not at random. Learn practical techniques to clean and impute missing values, build predictive models to estimate missing entries, and preserve essential data during ETL transformations.

When combining data from multiple sources, we often find errors, inconsistencies, and inaccuracies. During the data cleaning step, we focus on finding and removing duplicate data and handling missing values.

Both of these operations might sound easy in theory, but in practice, they require a great deal of business context and understanding of the data and business before we can take any action.

Handling missing values

When we encounter missing values, the first thing we should do is ask ourselves why the data is missing. How much is missing? And can we fill it?

There could be countless reasons why data is missing:

- Human error (accidental deletion)
- Technical errors (blackouts in the middle of updating a DB)
- Even a thunderstorm near a sensory device

Ask

The easiest thing to do when a value is missing is to remove the entire record or column. However, the data might be important, and we should keep as much of it

as possible. If we could figure out why the data went missing, we might be able to salvage it and fill it with another value.

Types of missing values

Generally, there are three types/reasons why a data point could be missing:

Missing completely at random (MCAR)

This refers to a situation where the probability that a data point is missing is independent of the values of the missing data point and any other variables in the dataset. The missing data is not related to any pattern or structure in the data. If a data point is truly missing completely at random, we should either drop it or replace it with the mean, median, or mode values.

For example, a speed radar produced a missing value because of an earthquake. It had nothing to do with the car (other variables) or its speed (missing value). It was completely random.

Sample data collected by the speed radar:

cars_data_exampl... C++

Ask

Missing speed values are MCAR

Click the “Run” button in the following script to load the CSV file and drop missing values using Apache Spark:

main.py Python 3.10.4

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("MCAR").getOrCreate()

# Loading the CSV file
print('\nLoading the data')
df = spark.read.option('header', 'true').option('inferSchema', 'true')\
    .csv('/usercode/tmp/cars_data_example1.csv')

# Drop the rows with missing values
print('\nRemoving missing values...')
df_dropped = df.na.drop()
```

Run

Dropping missing values

- **Line 4**: We create a spark session.
- **Line 8:** We load the CSV file.
- **Line 13:** We remove missing values.
- **Line 18:** We print the new DataFrame.

Missing at random (MAR)

Refers to a situation where the probability that a data point is missing is dependent on one or more other variables/columns in the dataset but not on the value of the missing data point itself.

Ask

For example, a speed radar produced a missing value because it can’t detect black cars. It has something to do with the car’s color (other variables) but not its

speed (missing value).

If we have some domain knowledge, we might be able to salvage MAR data. For example, if we know black cars drive, on average, 25% faster than other cars, we could fill the missing values of all black cars with the average of the other cars multiplied by 1.25.

Sample of the data collected by the speed radar:

cars_data_exampl... C++

Missing speed values for black cars

Click the “Run” button in the following script to load the CSV file and replace the missing values for black cars with 1.25 times the average of the rest.

main.py Python 3.10.4

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("MAR").getOrCreate()

# Loading the CSV file
```

Ask

```
print('\nLoading the data')
df = spark.read.option('header', 'true').option('inferSchema', 'true')\
    .csv('/usercode/tmp/cars_data_example2.csv')

# Calculating avg speed for non-black cars
avg_speed = round(df.filter(df['Color'] != 'Black').agg({'Speed':'mean'})
```

Run

Replacing missing values with the mean

- **Line 4:** We create a spark session.
- **Line 8:** We load the CSV file.
- **Line 12:** We calculate the average speed of all non-black cars and round output up to three decimal points.
- **Line 17:** We impute missing values with the average speed of all non-black cars times 25%.
- **Line 21:** We print the new DataFrame.

Missing not at Random (MNAR)

In this situation, the probability that a data point is missing depends on the value of the missing data point. In other words, the data is missing because of its value.

For example, a speed radar produced a missing value because it sometimes cannot detect cars over 200 km/h. It has nothing to do with the car itself (other variables). It happened because of the speed (missing value).

We should not delete the MNAR data because the "missingness" of the object is what makes it interesting. However, it is also the most difficult to deal with. To handle this, we need to understand why it is missing, and we might need to analyze the rest of the observed data and estimate the missing data based on the observed data, using a statistical model like maximum likelihood estimatio (MLE).

Ask

We could create a linear regression model that predicts the speed of cars with missing speeds. The model can consider variables like the car's manufacturer and the year it was made.

The model will estimate the speed of vehicles with missing speed data. If the predicted speed exceeds 200 km/h, we can replace the missing value with 200. On the other hand, if the predicted speed is less than 200 km/h, we can discard the corresponding record.

Sample data from the speed radar:

cars_data_exampl... C++

Missing speed values because cars passed 200 km/h

Creating the training data

To create the linear regression model, we initially have to train it using some available data. The model will generate a simple equation to describe the relationship between the `Make` and `Year` of a car and its recorded `Speed`. Th used for training will be from all vehicles for which speed data is available.

Ask

main.py Python 3.10.4

```
from pyspark.sql import SparkSession

# Create a Spark session
spark = SparkSession.builder.appName("MNAR").getOrCreate()

# Loading the CSV file
print('\nLoading the data')
data = spark.read.option('header', 'true').option('inferSchema', 'true')\
  .csv('/usercode/tmp/cars_data_example3.csv')

# removing missing values to create the training data
print('\nCreating the training data')
training_data = data.filter(data.Speed.isNotNull()).select('Make', 'Year'
```

Run

Creating the training data

- **Line 13:** We create training data by removing rows with missing `Speed` and selecting only the two columns, `Make` and `Year`.

Building the model

Next, we'll preprocess the training data, such as converting categorical/string variables (like `Make`) into numerical ones, and then build the linear regression model.

main.py Python 3.10.4

```
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.regression import LinearRegression

# Convert categorical variables to numerical values
print('\nConverting categorical variables into numerical')
indexer_make = StringIndexer(inputCol="Make", outputCol="MakeIndex")
indexed_make = indexer_make.fit(training_data).transform(training

# Assemble the features into a single vector column
print('\nAssembling the data into a single vector')
assembler = VectorAssembler(inputCols=["MakeIndex", "Year"], outputCol="f
```

Ask

```
assembled = assembler.transform(indexed_make)
```

Run

Building the linear regression model

- **Lines 6–7:** We convert the `Make` column values from strings to numerical values.
- **Lines 11–12:** We reduce the data of the `Make` and `Year` columns into a single numerical vector.
- **Line 16:** We set the variable to predict to be `Speed`.
- **Lines 20–21:** We create a linear regression model and train it.

Predicting the speed of cars with missing values

Finally, we need to collect the data of cars with missing speed values, predict their speed, and handle the original dataset according to the results. The data we need to predict will be all the cars without a speed value.

main.py Python 3.10.4

```
# Collecting the data to predict
data_to_predict = data.filter(data["Speed"].isNull()).select('Make', 'Yea
data_to_predict.show()
```

Ask

Collecting the data to predict

So, there are four records with missing data. Click the “Run“ button in the following widget to see the speed values our model predicts for these cars.

main.py Python 3.10.4

```
# Convert categorical variables to numerical values
print('\nConverting categorical variables into numerical')
indexed_make = indexer_make.fit(data_to_predict).transform(data_to_predic

# Assemble the features into a single vector column
print('\nAssembling the data into a single vector')
assembled_new = assembler.transform(indexed_make)

# Predict new data
print('\nPredicting speed values')
predictions = model.transform(assembled_new)

print('\nResults:')
```

Run

Predicting speed values for cars with missing data

Our model predicts that only a 2023 Tesla's speed will be above 200 km/h. As a result, we should substitute the missing speed value in the original data set with 200 for this vehicle and discard the rest of the cars with missing speed values.

Ask